

Advanced FPP Modeling

Robert Bocchino
NASA Jet Propulsion Laboratory
February 12, 2026

Copyright © 2026 California Institute of Technology.
Government sponsorship acknowledged.

Jet Propulsion Laboratory
California Institute of Technology

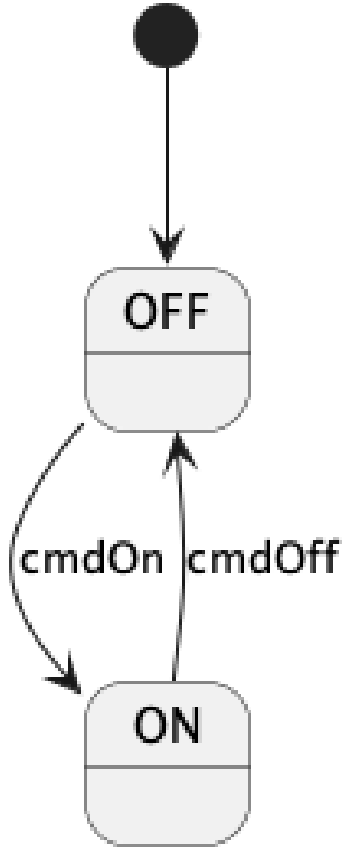
Introduction

- **F Prime Prime** or **FPP** is the modeling language of F Prime
- This course assumes basic knowledge of FPP
  - Types and ports
  - Components and component instances
  - Topologies
- In this section of the course, we will cover the following topics:
  - **State machines:** A way of specifying states and transitions
  - **Subtopologies:** Hierarchical groups of instances and connections
  - **Custom analysis and code generation:** Writing analyzers and code generators

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

State Machines

- A **state machine** is a way of describing a software system
  - **State:** A mode of system behavior
  - **Transition:** A change from one state to another
  - **Signal:** An input causing a transition
- Example: A simple switched device
  - Two states: OFF and ON
  - The initial state is OFF
  - Two signals: cmdOn and cmdOff
  - Each signal causes the corresponding transition



Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

State Machines in FPP

- FPP provides two ways to model state machines
  - **External state machines:** Define the state machine in another tool
  - **Internal state machines:** Define the state machine in FPP
- External state machines were a first version of state machines in FPP
  - Designed to support a tool called **STARS**
  - STARS now supports generation of internal state machines
- Here we'll cover internal state machines
  - Based on a subset of **statecharts** in the Universal Modeling Language (UML)
  - Can be rendered using PlantUML (https://plantuml.com)

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

The Device State Machine in FPP

```
@ A state machine representing a device with on-off behavior
state machine Device {

  @ A signal for turning the device on
  signal cmdOn

  @ A signal for turning the device off
  signal cmdOff

  @ The initial state is OFF
  initial enter OFF

  @ The ON state
  state ON {

    @ In the ON state, a cmdOff signal causes a transition to the OFF state
    on cmdOff enter OFF

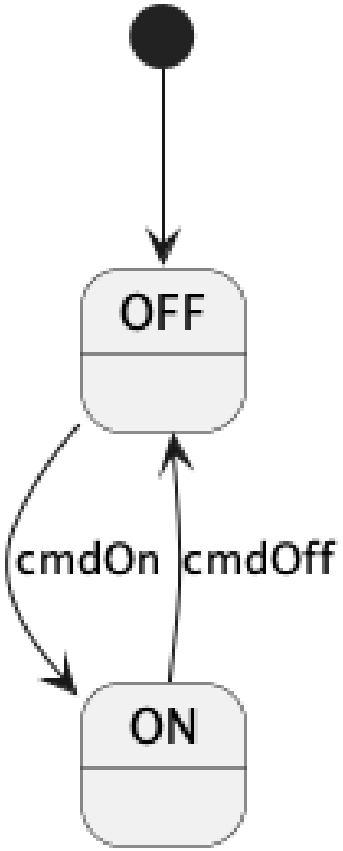
  }

  @ The OFF state
  state OFF {

    @ In the OFF state, a cmdOff signal causes a transition to the ON state
    on cmdOn enter ON

  }

}
```



Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Instantiating State Machines

- An FPP state machine definition generates an abstract base class
- You can implement the base class in two ways
  - Write a direct implementation
  - Specify a state machine instance in component definition
- The second way has more autocode support
- The first way supports very general uses of FPP state machines
  - E.g., you could implement a fault monitor this way
  - The monitor could be a C++ class, not an F Prime component

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

State Machine Instances in Components

```
@ An external state machine
state machine M

@ A component with a state machine
active component StateMachine {

  @ A port for driving the state machine
  async input port schedIn: Svc.Sched

  @ An instance of state machine M
  state machine instance m: M priority 10 drop

}
```

Components with state machines must be active or queued
Signals to state machine instances go on an internal queue

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Actions

```
@ A state machine representing a device with on-off behavior,
@ with actions on transitions
state machine Device {

  @ An action on the transition from OFF to ON
  @ The value counts the number of times this action has occurred
  action offOnAction: U32

  @ A signal for turning the device on
  @ The value counts the number of times this signal has been received
  signal cmdOn: U32

  @ A signal for turning the device off
  signal cmdOff

  @ The initial state is OFF
  initial enter OFF

  @ The ON state
  state ON {

    @ In the ON state, a cmdOff signal causes a transition to the OFF state
    on cmdOff enter OFF

  }

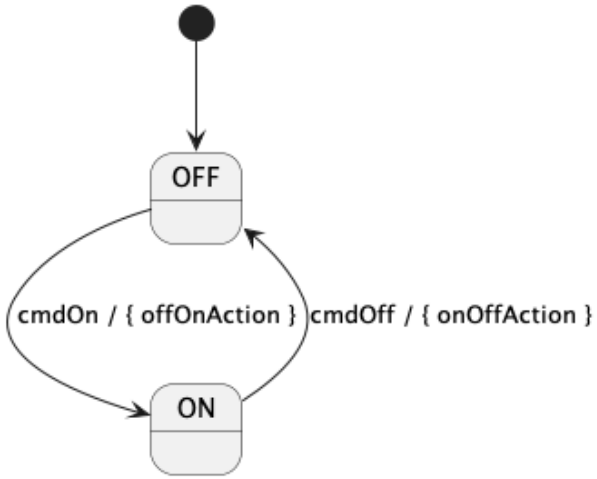
  @ The OFF state
  state OFF {

    @ In the OFF state, a cmdOff signal causes a transition to the ON state
    @ Before entering the ON state, do offOnAction, passing the data from
    @ the signal into the action
    on cmdOn do { offOnAction } enter ON

  }

}
```

```
@ In the OFF state, a cmdOff signal causes a transition to the ON state
@ Before entering the ON state, do offOnAction, passing the data from
@ the signal into the action
on cmdOn do { offOnAction } enter ON
```



Actions that run on transitions can carry data
The data is provided by the signal
Actions can also occur on entry and exit

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Guards

```
@ A device state machine with a guarded transition
state machine Device {

  @ A guard for checking whether the device is in a safe state for power-on
  guard powerOnIsSafe

  @ A signal for turning the device on
  signal cmdOn

  @ A signal for turning the device off
  signal cmdOff

  @ The initial state is OFF
  initial enter OFF

  @ The ON state
  state ON {

    @ In the ON state, a cmdOff signal causes a transition to the OFF state
    on cmdOff enter OFF

  }

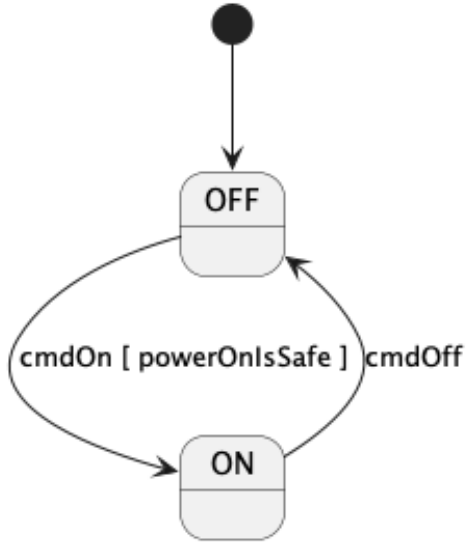
  @ The OFF state
  state OFF {

    @ In the OFF state, a cmdOff signal causes a transition to the ON state
    @ if powerOnIsSafe evaluates to true. Otherwise no transition occurs.
    on cmdOn if powerOnIsSafe enter ON

  }

}
```

```
@ In the OFF state, a cmdOff signal causes a transition to the ON state
@ if powerOnIsSafe evaluates to true. Otherwise no transition occurs.
on cmdOn if powerOnIsSafe enter ON
```



A guard is a predicate function
A guarded transition occurs if the function evaluates to true
Like an action, a guard can have a type

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Choices

```
@ A device state machine with a choice
state machine Device {

  @ A guard for checking whether the device is in a safe state for power-on
  guard powerOnIsSafe

  @ A signal for turning the device on
  signal cmdOn

  @ A signal for turning the device off
  signal cmdOff

  @ A signal for resetting the device
  signal cmdReset

  @ The initial state is OFF
  initial enter OFF

  @ The ON state
  state ON {

    @ In the ON state, a cmdOff signal causes a transition to the OFF state
    on cmdOff enter OFF

  }

  @ The OFF state
  state OFF {

    @ In the OFF state, a cmdOff signal causes a transition to the choice ON_OR_ERROR
    on cmdOn enter ON_OR_ERROR

  }

  @ The ON_OR_ERROR choice
  choice ON_OR_ERROR {
    if powerOnIsSafe enter ON else enter ERROR
  }

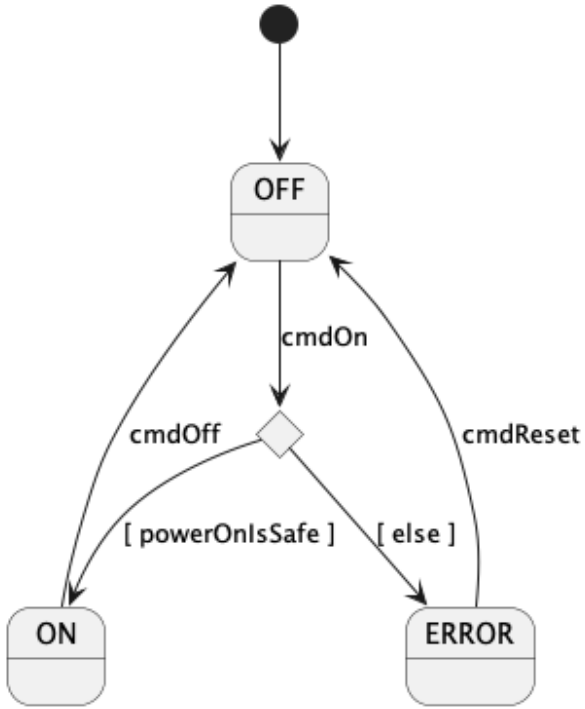
  @ The ERROR state
  state ERROR {

    @ In the ERROR state, a cmdReset signal causes a transition to the OFF state
    on cmdReset enter OFF

  }

}
```

```
@ The ON_OR_ERROR choice
choice ON_OR_ERROR {
  if powerOnIsSafe enter ON else enter ERROR
}
```



A choice is a branching transition

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Hierarchy

```
@ The ON state
state ON {

  @ In the ON state, a cmdOff signal causes a transition to the OFF state
  on cmdOff enter OFF

  @ In the ON state, a cmdUnsafe signal causes a transition to OFF.UNSAFE
  on cmdUnsafe enter OFF.UNSAFE

}

@ The OFF state
state OFF {

  @ The initial state is SAFE
  initial enter SAFE

  @ The state OFF.SAFE
  @ In this state, it is safe to turn on the device
  state SAFE {

    @ In the SAFE state, a cmdOff signal causes a transition to the ON state
    on cmdOn enter ON

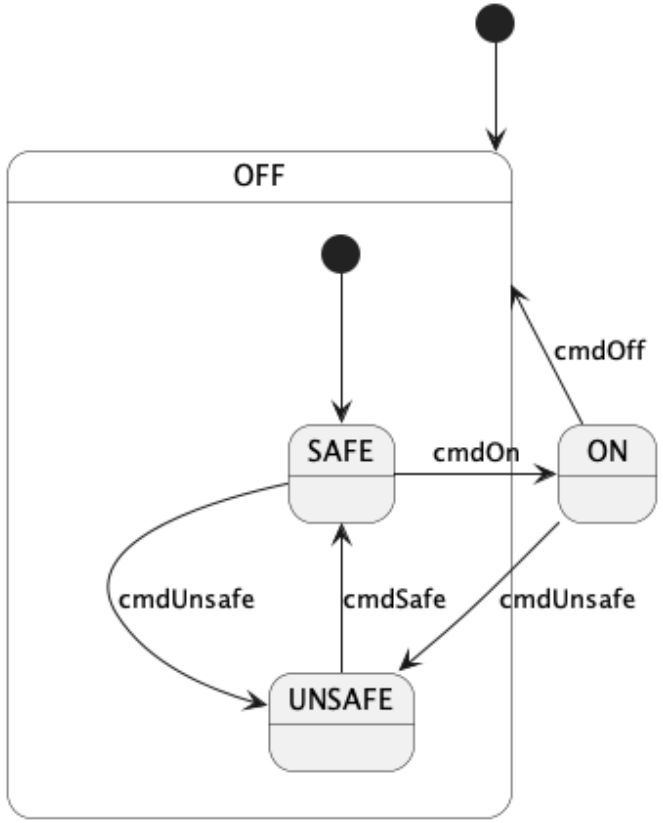
    @ In the SAFE state, a cmdUnsafe signal causes a transition to the UNSAFE state
    on cmdUnsafe enter UNSAFE

  }

  @ The state OFF.UNSAFE
  @ In this state, it is not safe to turn on the device
  state UNSAFE {

    @ In the UNSAFE state, a cmdSafe signal causes a transition to the SAFE state
    on cmdSafe enter SAFE

  }
```



Hierarchy means states within states
See the FPP User's Guide for details

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Subtopologies

- Often, it is useful to decompose a FSW system into several topologies
- For example:
  - Command and data handling (CDH)
  - Various subsystems: power, thermal, attitude control, etc.
  - An overall system topology
- Each of the subsystem topologies might depend on CDH
- The system topology might depend on each of the subsystems
- The lower-level topologies are called **subtopologies**

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Importing Topologies in FPP

```
topology B {

  import A

  ...

}
```

- How this works:
  - Resolve *A* to a topology *T*
  - Take the union of the instances in *B* and *T*
    - If *B* and *T* each have an instance *a*, that's one instance
  - Take the **disjoint union** of the connections in *B* and *T*, disregarding port numbers
    - If *B* and *T* each have a connection *a.b -> c.d*, that's two connections
  - Resolve *B*
    - Resolve pattern graphs by adding arrows
    - Resolve port numbers, respecting numbers already there

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Import Resolution: Example

```
topology A {

  instance a
  instance b

  connections C1 {
    a.p1 -> b.p
  }

}

topology B {

  import A

  instance c

  connections C1 {
    a.p1 -> c.p
  }

  connections C2 {
    a.p2 -> c.p
  }

}
```



```
topology B {

  instance a
  instance b
  instance c

  connections C1 {
    a.p1 -> b.p
    a.p1 -> c.p
  }

  connections C2 {
    a.p2 -> c.p
  }

}
```


Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Setup and Teardown Phases

- Setup of an F Prime deployment is divided into **phases**
  - Declare instances, initialize instances, register commands, etc.
  - Teardown is similar
- When defining a component instance, you can specify code to run
  - When to run it (what phase)
  - What to run (C++ code snippet)
- This is useful in working with subtopologies
- The code generator includes all the code for each instance

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Setup and Teardown Phases: Example

```
instance cmdSeq: Svc.CmdSequencer base id 0x0700 \
  queue size Default.queueSize \
  stack size Default.stackSize \
  priority 100 \
{

  phase Fpp.ToCpp.Phases.configConstants """
  enum {
    BUFFER_SIZE = 5*1024
  };
  """

  phase Fpp.ToCpp.Phases.configComponents """
  cmdSeq.allocateBuffer(
      0,
      Allocation::mallocator,
      ConfigConstants::SystemReference_cmdSeq::BUFFER_SIZE
  );
  """

  phase Fpp.ToCpp.Phases.tearDownComponents """
  cmdSeq.deallocateBuffer(Allocation::mallocator);
  """

}
```


Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Upcoming Features

- Topology ports
  - Topologies can have ports that map to ports of component instances
  - Then subtopologies can be connected to each other
- Module templates
  - You can specify parts of a module (constants, topologies, etc.) as a template
  - You can instantiate the template multiple times
    - For example, you can have several topologies with a similar structure
    - The templates can have parameters
  - For now, use syntactic inclusion to replicate modules

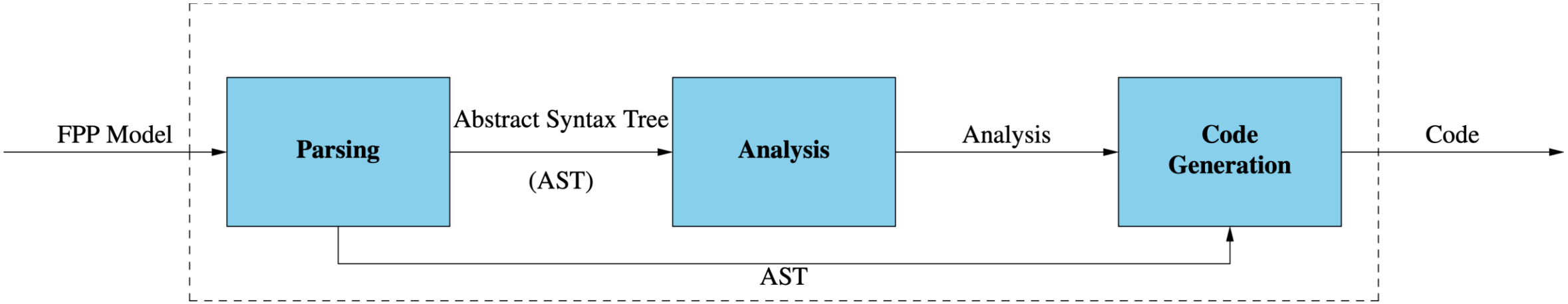
Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Custom Analysis and Code Generation

- FPP comes with several standard tools. For example:
  - *fpp-to-cpp*: Generate C++ implementation code
  - *fpp-to-dict*: Generate JSON dictionary code
  - *fpp-to-json*: Generate JSON code for the whole model
- Each of these tools
  - Performs a standard analysis on the model
  - Does standard code generation
- Sometimes you need to write your own tools. For example:
  - Check that a topology conforms to project-specific requirements
  - Generate some code to perform project-specific validation at run time

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

How Analysis and Code Generation Works



Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Abstract Syntax Tree (AST)

```
constant a = 0
constant b = a + 1
```



```
def constant
  ident a
  literal int 0
def constant
  ident b
  expr binop
    ident a
    binop +
    literal int 1
```


Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Developing Custom Tools

- FPP is implemented in Scala (https://www.scala-lang.org)
  - Functional Scala is great for parsing, analysis, and translation
  - See https://github.com/nasa/fpp/wiki/Pure-Functional-Programming-in-Scala
  - Many developers may be more comfortable using Python
- You have two choices
  - Develop in Scala
    - Create a new git repo
    - Add the FPP repo (https://github.com/nasa/fpp) as a subrepository
  - Develop in Python
    - Run *fpp-to-json* on the FPP model
    - Feed the output to the F Prime Python model (https://github.com/fprime-community/fprime-python-model)

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Scala vs. Python

FPP Syntax

[1, 2, 3]




```scala
sealed trait Expr
final case class ExprArray(elts: List[AstNode[Expr]]) extends Expr
```




```python
class Expr(ABC):
    pass


@dataclass
class ExprArray(Expr):
    """
    Array expression

    :param elts: List of expression AST nodes
    :type elts: List[AstNode[Expr]]
    """

    elts: List[AstNode[Expr]]
```

The Python implementation closely follows the Scala implementation

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Custom Analysis

- Recommended strategy
  - Run standard FPP parsing and analysis
  - Run some more analysis on the AST and analysis data structure
- Standard analysis is documented here: https://github.com/nasa/fpp/wiki/Analysis
  - It uses a standard pattern for visiting the AST and computing analysis data
  - The Python model also follows this pattern
- You can
  - Check custom properties of the model, e.g., how instances are connected
  - Leverage FPP annotations to specify new model elements

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.



Custom C++ Code Generation

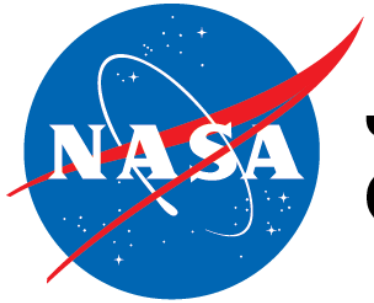
- The FPP code generators use the following pattern:
  - Visit the AST and build a data structure called *CppDoc*
  - Ask CppDoc to write itself as *.hpp* and *.cpp*
  - This simplifies the code generation
- You can use the same pattern in Scala or Python
- Docs and example use cases are in process

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Custom JSON Code Generation

- The FPP tools use a library called Circe to generate JSON
  - https://circe.github.io/circe/
  - In Scala you can use this library
  - In Python you can use the *json* module
- Currently, the best example is the FPP dictionary generator
  - https://github.com/nasa/fpp/tree/main/compiler/lib/src/main/scala/codegen/DictionaryJsonWriter
- Docs and examples are in process

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.



Jet Propulsion Laboratory
California Institute of Technology